

TESTING

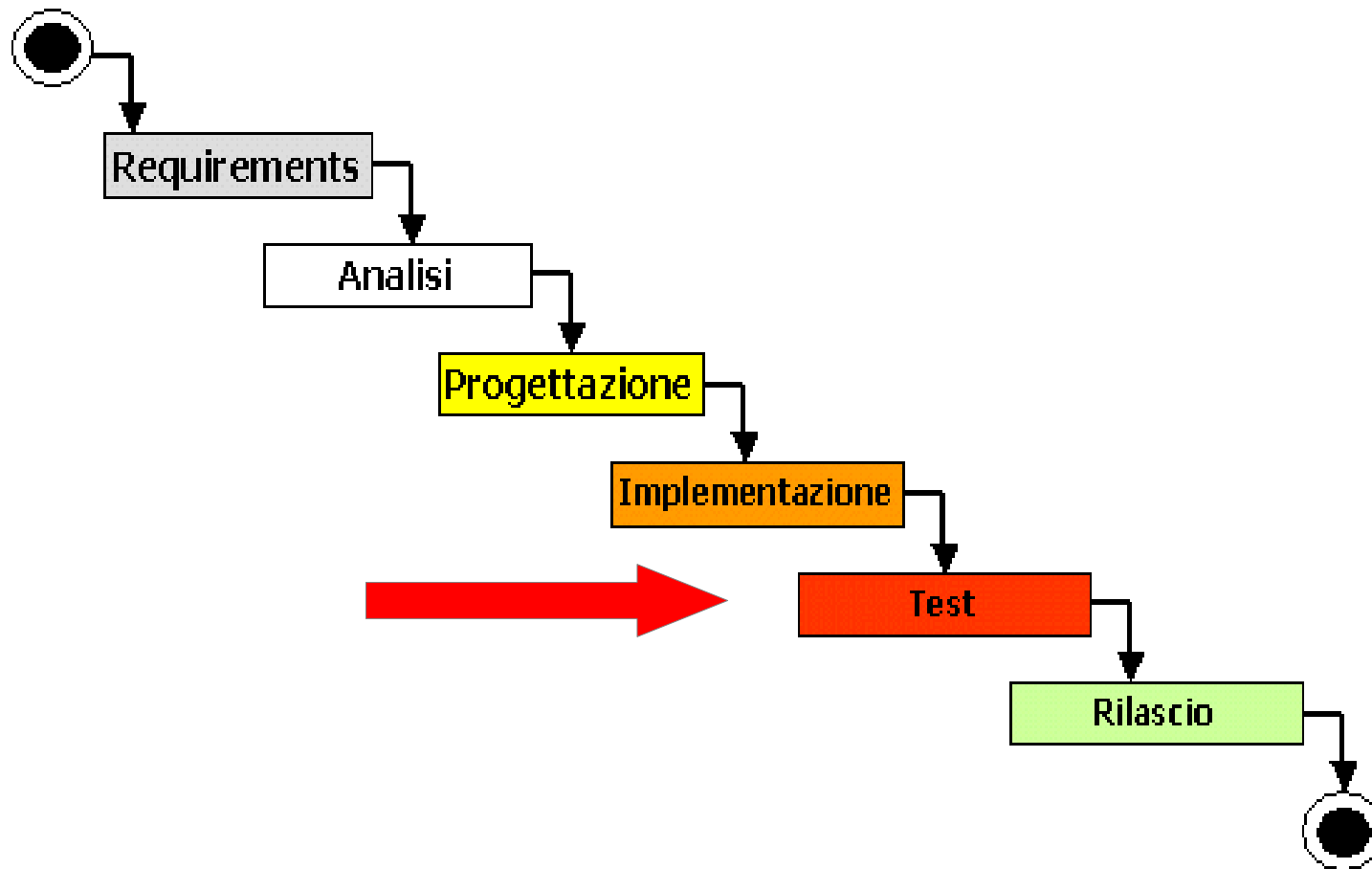
Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**

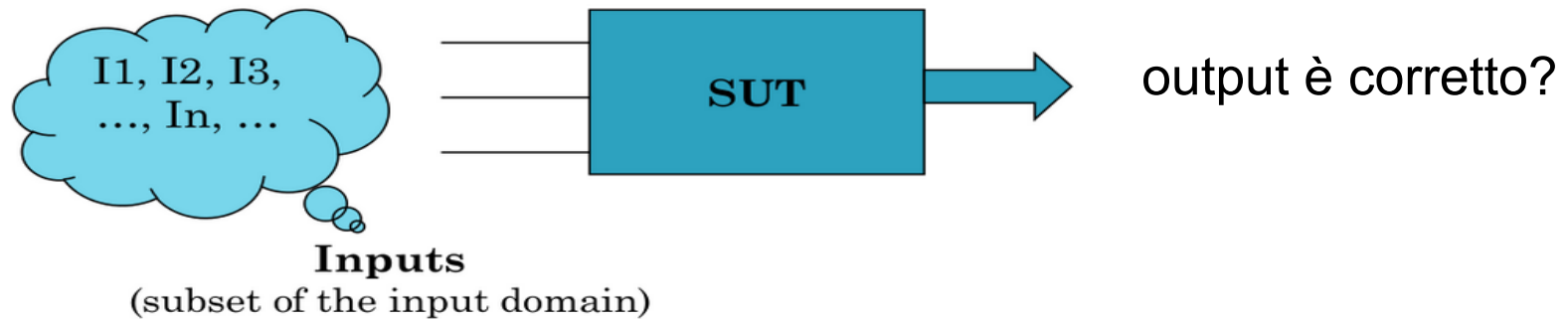


TESTING (nel Waterfall)



Cosa è il Software Testing ?

È una procedura sistematica che prevede l'esecuzione di un sistema software (System Under Test – SUT) con l'intento di trovare difetti (faults)



Risultato atteso = Risultato ottenuto?

Definizione IEEE

- *Il Software Testing è il processo di analisi di un prodotto software per rilevare le differenze tra le condizioni esistenti e richieste (cioè, gli errori) e per valutare le caratteristiche del prodotto software*



Definizione IEEE

Risultati corretti e non corretti

- ***Mistake (Errore Umano)*** – un'azione “umana” che produce un risultato non corretto
- ***Fault [or Defect] (Difetto)*** – un passaggio o processo o definizione dati non corretti in un programma
- ***Failure (Fallimento)*** – l'incapacità del sistema o di un suo componente di svolgere una funzione richiesta entro specifiche prestazioni desiderate.
- ***Error (Errore)*** – la differenza tra un valore o condizione calcolati, osservati o misurati e un valore o condizione reali, desiderati, teoricamente corretti.



Testing: Verifica e Validazione

- Il testing è in relazione con la **Verifica e Validazione (V&V)** del software
- **Verifica** (prima V) è *il processo di valutazione di un sistema o componente per determinare se il prodotto di una specifica fase di sviluppo soddisfa le condizioni imposte all'inizio di questa fase (IEEE 1990)*
 - Assicura che il software implementi correttamente specifiche funzioni
 - *Are we building the product right?*
 - *Stiamo implementando **correttamente il prodotto?***



Testing: Verifica e Validazione

- **Validazione** (la seconda V) è *il processo di valutazione di un sistema o componente durante o alla fine del processo di sviluppo per determinare se soddisfa specifici requisiti (IEEE 1990)*
 - Assicura che il prodotto soddisfi le richieste del cliente
 - *Are we building the right product?*
 - *Stiamo implementando **il prodotto corretto?***
 - La validazione è un concetto più difficile della verifica (che controlla se le specifiche iniziali sono soddisfatte) – essa ha a che fare con la correttezza delle specifiche rispetto alle reali esigenze del cliente.



Testing: Verifica e Validazione

- Differenze principali

	VERIFICA	VALIDAZIONE
Focalizzata su	Specifiche e progetto	Bisogni e aspettative dell'utente
Domanda guida	Stiamo implementando correttamente il prodotto?	Stiamo implementando il prodotto corretto?
Tipi di errore	Errori tecnici, logici	Errori nei requisiti o funzionali
Quando	Durante lo sviluppo	Alla fine o con prototipi



Scopi del Software Testing

- Verificare la qualità generale del software
- Trovare “bug” (difetti) nelle applicazioni
- Verificare la rispondenza alle specifiche, e la congruità delle stesse
- Verificare l'integrazione tra i sistemi
- Verificare che **modifiche fatte** non abbiano effetti non voluti (**test di regressione**)



Importanza del Software Testing

- Riduce i costi di sviluppo dell'applicazione:
 - Stime comuni indicano che un problema non riscontrato/inosservato fino alla produzione può essere da 40 a 100 volte più costoso rispetto alla sua risoluzione durante le fasi di sviluppo.
 - Assicura che l'applicazione si comporti esattamente come previsto
 - Riduce il costo totale di manutenzione per gli utenti finali
 - Il software ben testato si traduce in una maggiore soddisfazione degli utenti



Il Testing nel processo di sviluppo

- I test vanno eseguiti **il prima possibile e quante più volte** possibile:
- I test devono essere ripetibili e riusabili, possibilmente ***automatici***, cioè eseguibili come codice
- La pianificazione e l'esecuzione dei test è un processo iterativo:
 - Devono essere scritti appena si hanno a disposizione i requisiti
 - Devono essere aggiornati ogni volta che i requisiti cambiano



Testing esaustivo

- Testing esaustivo: testare un programma con tutti gli input dell'*input domain* (insieme di tutte le combinazioni possibili di input)
- P: I-domain  output
- Però spesso l'input domain è molto grande: cresce esponenzialmente!
 - ***Il testing esaustivo non è usabile in pratica!***
- Servono delle strategie per trovare “buoni input”:
 - Input selection



Test case e Test suite

- Definizioni di test case (caso di test):
 - Un **Test case** è una sequenza di passi per testare se il comportamento di una funzionalità/aspetto di un'applicazione è corretto. Un test case deve avere la descrizione degli input e del comportamento atteso
 - Un **Test case** contiene una serie di input, precondizioni e risultati attesi sviluppati per un particolare obiettivo, come esercitare un particolare percorso di programma o per verificare la conformità con uno specifico requisito
 - Anche chiamati **test data**
- **Test suite** = collezione di Test case



Test case e Test suite

Che cosa è “concretamente” un test case dipende dal contesto:

Test case per Web Apps:

Input:

1. login alla pagina <http://www.abc.it/home.html>
2. vai sulla pagina “acquisti”
3. seleziona prodotto X
4. aggiungi al carrello

Output atteso:

Il prodotto X è stato aggiunto al carrello

sequence of steps
+
input
+
output

Test case per “sort”:

Ordine crescente

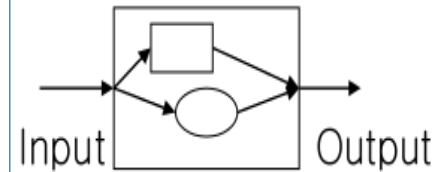
- Input: <“C”, 12, -29, 32, >
- Output atteso: -29 12 32

input
+
output



Due “Macro” Categorie

- Il **white box testing** è basato su una conoscenza esplicita del SUT (System Under Test) e della sua struttura
 - Chiamato anche structural testing
 - Basato sul concetto di “copertura”
 - *Statement coverage testing* è un esempio



- Il **black box testing** non è basato su una conoscenza del SUT e della sua struttura
 - Chiamato anche functional testing
 - Di solito gli input sono generati a partire dai requisiti/specifiche



White Box testing

- Il **White-box testing** è una tecnica di testing basata sull'analisi della struttura interna del codice.
- **Sinonimi:** Structural testing, Clear box testing, Glass box testing
- **Obiettivo:** Verificare come funziona il software al suo interno, non solo cosa fa (è un'attività di verifica).
- **Chi lo esegue:**
 - Il tester ha diretto accesso al codice
 - Spesso il tester coincide con lo sviluppatore stesso



White Box testing

- **Caratteristiche:** consente di
 - Verificare tutti i percorsi di esecuzione indipendenti
 - Eseguire tutte le condizioni logiche, sia true che false
 - Eseguire tutti i loop, inclusi i casi limite (off-by-one errors)
 - Controllare le strutture dati interne
- Gli **unit test** (tipico esempio di white-box testing):
 - Testano le singole unità del codice (funzioni, metodi, moduli)
 - Rilevano fino al 65% dei bug comuni



Copertura

- Il white-box testing si basa sul concetto di **copertura del codice**.
- Copertura (coverage) = percentuale del codice effettivamente eseguito durante i test.
- Esempio: Statement Coverage Testing
 - Assicura che ogni istruzione del codice venga eseguita almeno una volta.
 - *Non garantisce di coprire tutte le possibili combinazioni!*
 - È il tipo di copertura più semplice.
 - Fa parte dell'attività di verifica (non validazione).



White BoxTesting

- Con i white box test è possibile:
 - Garantire che tutti i percorsi indipendenti di un modulo siano eseguiti almeno una volta (**copertura!**)
 - Eseguire tutte le decisioni logiche (sia true che false)
 - Eseguire tutti i loop **compresi gli estremi** (errori *Obi Wan*, ovvero “off-by-one”):
 - Fare il check delle strutture interne di dati
- Gli unit test permettono di catturare circa il 65% dei bug



White-Box Testing: Limiti

- Spesso il numero di casi di test prodotti è molto grande
 - Non usabile in pratica in modo esaustivo!
- Per questo motivo è spesso usato solo per *Unit testing* e per alcune porzioni del sistema
 - **“Testing in the small”**
- Non è in grado di rivelare i fallimenti dovuti a **“missing feature errors”**
 - Possono essere scoperti solo considerando i requisiti/specifiche



Black Box testing

- Il **black box testing**, chiamato anche **functional testing** è una tecnica di test in cui il tester non conosce l'implementazione interna del sistema.
- Si basa su input, output e requisiti funzionali.
- E' un'attività di Verifica, ma può anche essere di Validazione:
 - Il tester non ha accesso al codice, il codice è una **black box**
 - Si basa sui requisiti
 - Non si guarda come funziona dentro, ma cosa restituisce fuori.



Black Box testing

- **Obiettivo:** testare ciò che il software fa, verificando se si comporta secondo le specifiche.
- **Caratteristiche:**
 - Il tester guarda solo l'input e l'output.
 - Non ha accesso alla logica interna, né al codice.
 - Si basa su requisiti funzionali.
- **Vantaggi:**
 - Simula il comportamento reale dell'utente finale.
 - Può essere eseguito anche da chi non è sviluppatore.
 - Utile per test di sistema, accettazione, UI, API.
- **Limiti:**
 - Non verifica la copertura del codice interno.
 - Può mancare difetti logici non visibili solo dall'esterno.



Black Box Testing

- Test Funzionali e Test Comportamentali
- Cerca di individuare 5 tipi di errori
 - Funzioni mancanti o non corrette
 - Errori nell'interfaccia
 - Errori nelle strutture dati usate dalle interfacce
 - Errori di prestazioni e di comportamento
 - Errori di inizializzazione e terminazione
- Attraverso questi test si determina se un modulo o un sistema è in accordo con le specifiche



Black Box Testing

- Si può applicare a qualsiasi tipo di sistema: Applicazioni Web, App per smartphone, Sistemi SOA/API, Sistemi embedded
- **Non** devono essere pianificati ed eseguiti dai programmatori che hanno scritto il codice
- I test possono essere progettati appena disponibili i requisiti, quindi **prima ancora che il codice sia scritto**
- Molte organizzazioni hanno gruppi di testing indipendenti e l'attività di test è spesso affidata a terze parti



Quale approccio è il migliore ?

- In assoluto non è possibile rispondere, dipende dal contesto
- In generale possiamo dire che **sono complementari e vanno adottati entrambi ...**
- Anche se esiste uno studio che afferma:
 - “Con tester esperti, il testing funzionale rivela più difetti del testing strutturale”



Livelli di Testing

Testing Type	Specification	Kind of Testing
Unit	Low-Level Design Actual Code Structure	White Box
Integration	Low-Level Design High-Level Design	White Box Black Box
Functional and System	High Level Design Requirements Analysis	Black Box
Acceptance	Requirements Analysis	Black Box
Beta	Ad hoc	Black box
Regression	Changed Documentation High-Level Design	Black Box White Box

+ **Stress Testing**

Robustness testing

Black Box



Livelli di Testing

- Ci sono 8 livelli di testing che possono essere eseguiti su un sistema software
- Divisi per tipologie: tipo di test e applicazione del test
- **Tipi di test:** Unit Testing, Integration Testing, Functional Testing, System Testing, Stress Testing. Descrivono il livello tecnico o il focal point del test.
- **Applicazione del test:** Acceptance Testing, Regression Testing, Beta Testing. Descrivono quando e perché eseguire quei test nel ciclo di sviluppo.
- Uno stesso tipo di test può essere usato in diversi contesti applicativi.



Livelli di Testing – Tipi di test

Unit testing è *il testing di singole unità hardware o software o gruppi di unità connesse (IEEE 1990)*

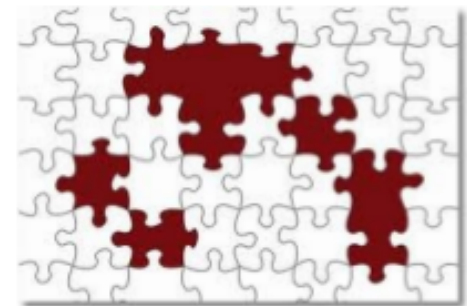
- L'unità (funzione, metodo, classe, ...) viene testata dallo sviluppatore isolandola il più possibile dal resto
- Concetto di **stub / mock object**: oggetti fittizi che simulano il contesto reale del test
- Di solito è un approccio *white-box* e automatizzabile



Livelli di Testing – Tipi di test

Integration testing *è il test in cui i componenti software, componenti hardware, o entrambi sono combinati e testati per valutare l'interazione tra di loro (IEEE 1990)*

- I moduli sono “testati assieme”
- Viene verificato come comunicano i moduli e quali informazioni vengono scambiate
- **Le interfacce** tra moduli di solito sono fonti di errori
 - Ordine sbagliato dei parametri
 - Precondizioni non rispettate
 -



Livelli di Testing – Tipi di test

Functional testing *è un tipo di testing che garantisce che le funzionalità descritte nelle specifiche dei requisiti siano rispettate*

- Può essere eseguito a vari livelli: Su singole funzioni (unità), su moduli integrati, su sistemi completi.
- E' un test black box.



Livelli di Testing – Tipi di test

System testing è *un test condotto su un sistema completo e integrato, per la valutazione della conformità del sistema con le sue esigenze specifiche (IEEE 1990)*

- Il System testing verifica che il software soddisfi i requisiti e anche i bisogni dell'utente (acceptance testing o test di accettazione)
- È un test *black-box*
- Non solo aspetti di correttezza ma anche *prestazioni, usabilità, security, ...*



Livelli di Testing – Tipi di test

Stress Testing:

- Consiste in test intensi o approfonditi utilizzati per determinare la stabilità di un determinato sistema, infrastruttura o entità critica.
- Implica test oltre la normale capacità operativa, spesso fino a un punto di rottura, al fine di osservare i risultati.
- Uno stress test di sistema cerca di verificarne la **robustezza**, la **disponibilità** e la gestione degli errori sotto un carico elevato.
- Sono test funzionali/di sistema, ma anche eseguiti con tecniche specifiche



Livelli di Testing – Applicazione dei test

Acceptance testing è *un test formale condotto per determinare se un sistema soddisfa i suoi criteri di accettazione (i criteri che il sistema deve soddisfare per essere accettato da un cliente) e per consentire al cliente di stabilire se accettare o meno il sistema (IEEE 1990)*

- È un **uso specifico** di test funzionali e/o di sistema
- È una forma di validazione: non interessa il codice, ma il comportamento.
- Spesso usa test funzionali o di sistema già pianificati.



Livelli di Testing – Applicazione dei test

Regression testing *si tratta di ripetere il test selettivo di un sistema o di un componente per verificare che le modifiche non abbiano causato effetti indesiderati e che il sistema o il componente soddisfi ancora i suoi requisiti (IEEE 1990)*

- Tutte le volte che si effettua una modifica in un applicazione c'è il rischio di *side effects* (effetti collaterali) in zone che apparentemente non sembrerebbero impattate dalla modifica
- Scopo del test di regressione è verificare che non ci siano stati side effect
- **Avere un *suite* di test automatici aiuta grandemente i test di regressione**



Livelli di Testing – Applicazione dei test

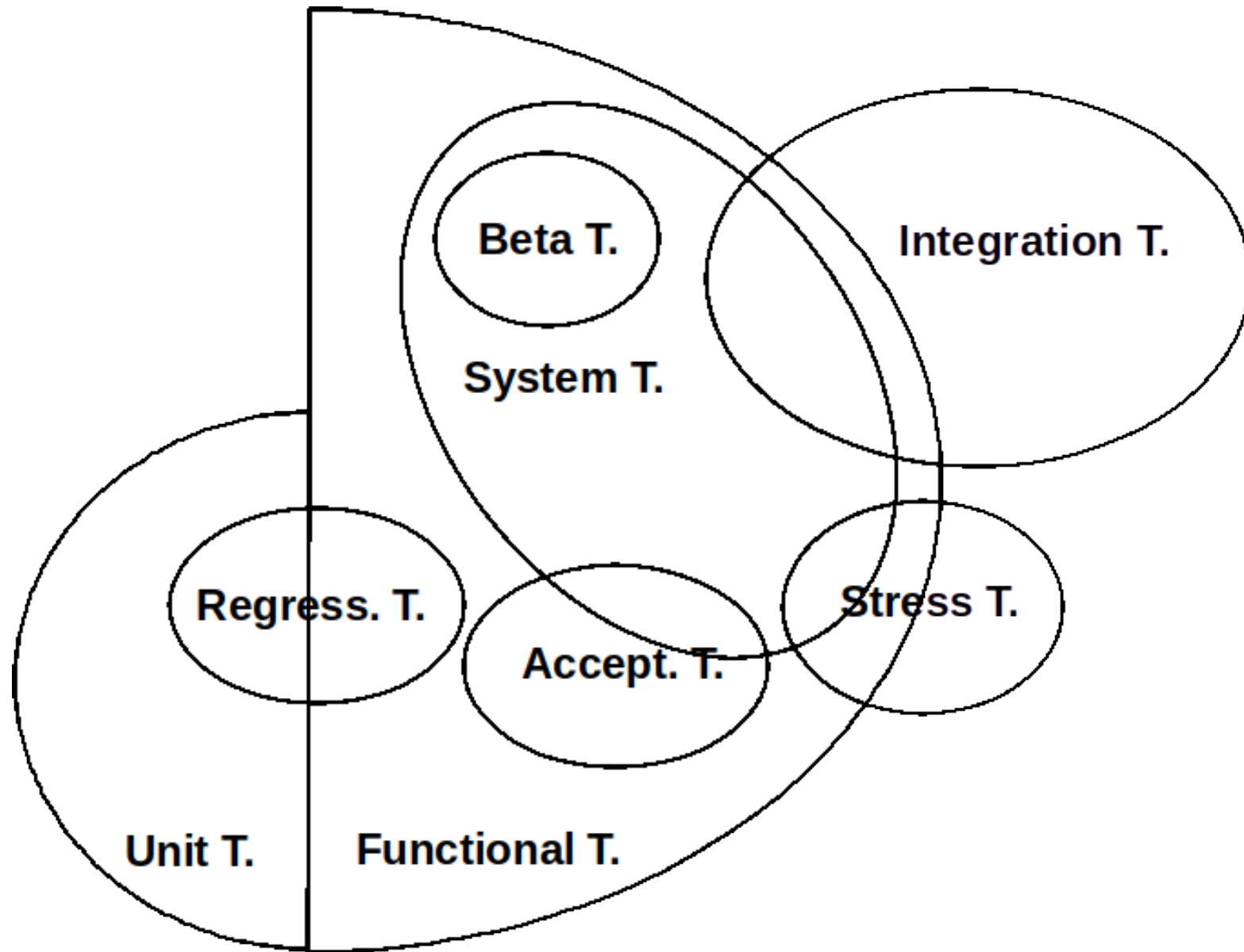
Beta Testing:

*Quando una versione avanzata parziale o totale di un pacchetto software è disponibile, e l'organizzazione di sviluppo la offre gratuitamente a uno o più (e a volte migliaia di) potenziali utenti (**beta tester**) per usarla, trovare errori e segnalarli per essere corretti*

- In tal modo, si raccolgono segnalazioni di difetti e feedback dagli utenti
- E' un system testing effettuato usando realmente il sistema



Diagr. di Venn dei Livelli di Testing



Piano di Test

- Un **piano di test** (*test plan*) è un documento che descrive lo scopo, l'approccio, le risorse e la pianificazione delle attività di test previsti. Esso individua gli elementi da testare, gli aspetti da testare, le attività di test, nonché eventuali rischi che richiedono piani di emergenza (IEEE, 1990)
- Un componente importante del test plan è l'individuazione dei **casi di test**



Test Planning

- Il *test plan* dovrebbe essere definito il prima possibile nel ciclo di sviluppo
- Il formato del *test case* è molto importante. Lo schema usato generalmente è il seguente:
 - **Test ID**: identificatore univoco del test
 - **Description**: passi da eseguire e condizioni di ingresso (precondizioni)
 - **Expected Results**: risultati attesi dal test
 - **Actual Results**: risultati ottenuti dal test



Test Planning

Test ID	Description	Expected Results	Actual Results



Test Planning

- I campi del test case devono essere compilati in modo chiaro e non ambiguo
- **Se si verifica un problema, deve essere possibile ricrearlo**
- I test case devono corrispondere ai requisiti espressi dal cliente

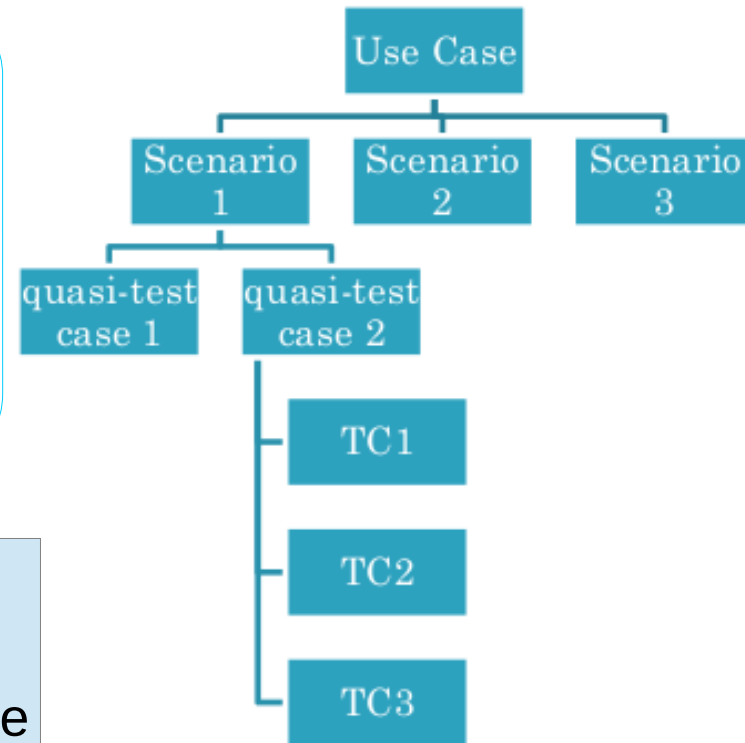


Approccio “THREE STEP”

Generazione dei casi di test a partire dai casi d'uso

- Per ogni caso d'uso, generare un'insieme completo di scenari
- Per ogni scenario, trovare le condizioni che permettono di eseguirlo (quasi-test case)
- Per ogni quasi-test case, identificare i valori di input e creare uno o più casi di test

Three-step approach
for generating test
cases from a use case



Test early and often

- Eseguire i test case il prima possibile è il modo migliore per ricevere feedback sul codice
- Per far girare i test case gli sviluppatori devono integrare i vari pezzi di codice in un codice comune del sistema, il più possibile completo
- Si raccomanda che il codice venga integrato una volta al giorno (*integrazione continua*)
- Dopo l'integrazione, si fanno girare tutti i *test automatici*



Esempio: Unit Test

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def get_area(self):
        return self.width * self.height

    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height
```



Esempio: Unit Test

```
import unittest

class TestGetAreaRectangle(unittest.TestCase):
    def test_normal_case1(self):
        rectangle = Rectangle(2, 3)
        self.assertEqual(rectangle.get_area(), 6, "wrong area")

    def test_normal_case2(self):
        self.rectangle.set_width(2)
        self.rectangle.set_height(4)
        self.assertTrue(rectangle.get_area() == 8, "wrong area")
```

